TP N°5 **Enseignante : Arij Abid**

TP5 Django – Gestion des Articles

On souhaite développer une application Django de gestion des articles.

1. Modèle de données

Dans le fichier models.py de cette application, créez le modèle Article.

Modèle Article: models.py

```
from django.db import models
from django.utils import timezone


class Article(models.Model):
    titre = models.CharField(max_length=100)
    contenu = models.TextField()
    date_pub =  models.DateTimeField(default=timezone.now)
    def __str__(self):
        return self.titre
```

2. Création des routes (URLs): Configurer les routes suivantes dans urls.py :

URL	Rôle
/liste	Afficher la liste des articles
/create	Ajouter un nouvel article
/update/<id>	Modifier un article
/delete/<id>	Supprimer un article
/article/<id>	Afficher le détail d'un article

```
path('liste', views.liste_articles, name='liste_articles'),
path('create', views.create_articles, name='create_articles'),
path('update/<int:pk>/', views.update_articles, name='update_articles'),
path('delete/<int:pk>/', views.delete_articles, name='delete_articles'),
path('article/<int:id>/', views.details_article, name='details_article'),
```

3. Installation de Django Crispy Forms

Utilisé pour rendre les formulaires plus beaux (Bootstrap 5).

pip install django-crispy-forms

pip install crispy-bootstrap5

4. Configuration dans settings.py

a) Ajouter dans INSTALLED_APPS

INSTALLED_APPS = [

'crispy_forms',

'crispy_bootstrap5',

]

b) Configurer le template pack par défaut

CRISPY_ALLOWED_TEMPLATE_PACKS = "bootstrap5"

CRISPY_TEMPLATE_PACK = "bootstrap5"

5. Formulaire Django (forms.py) :

Crée le fichier FirstApp/forms.py

forms.py

FirstApp > forms.py > ArticleForm > Meta

```
1  from django import forms
2  from .models import Article
3
4  class ArticleForm(forms.ModelForm):
5      class Meta:
6          model = Article
7      #   fields = ['titre', 'contenu']
8          fields = '__all__'
```

- **ModelForm:** permet de créer automatiquement un formulaire à partir d'un modèle (Article). Dans fields, tu choisis quels champs de ton modèle tu veux afficher dans le formulaire.

Différence entre les deux	fields = ['titre', 'contenu']	**fields = '__all__'**
	Tu choisis exactement quels champs du modèle seront affichés dans le formulaire.	Affiche automatiquement TOUS les champs du modèle (sauf les champs non éditables).

6. Création des vues Django

```
from django.shortcuts import render , redirect ,get_object_or_404
from django.http import HttpResponse
from .models import Article
from datetime import datetime
from .forms import ArticleForm
```

a) Afficher la liste des articles

- Récupérer tous les articles depuis la base de données
- Les envoyer au template liste_articles.html

```
def liste_articles(request):
    articles = Article.objects.all()
    return render(request, 'liste_articles.html', {'articles': articles})
```

b) Créer un article

- Afficher un formulaire
- Sauvegarder l'article après validation
- Rediriger vers la liste

```
# Create view.
def create_articles(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect('liste_articles')
    else:
        form = ArticleForm()
    return render(request, 'create_articles.html', {'form':form})
```

c) Modifier un article

- Récupérer l'article par son id
- Pré-remplir le formulaire

Sauvegarder les modifications

```
# updpate view
def update_articles(request, pk):
    article = Article.objects.get(id=pk)
    if request.method == 'POST':
        form = ArticleForm(request.POST, instance=article)

        if form.is_valid():
            form.save()
            return redirect('liste_articles')

    else:
        form = ArticleForm(instance=article)
    return render(request, 'update_articles.html', {'form':form})
```

d) Supprimer un article

- Confirmer la suppression
- Supprimer l'article
- Rediriger vers la liste

```
# delete view
def delete_articles(request, pk):
    article = Article.objects.get(id=pk)

    if request.method == "POST":
        article.delete()
        return redirect('liste_articles')
```

e) Détails d'un article

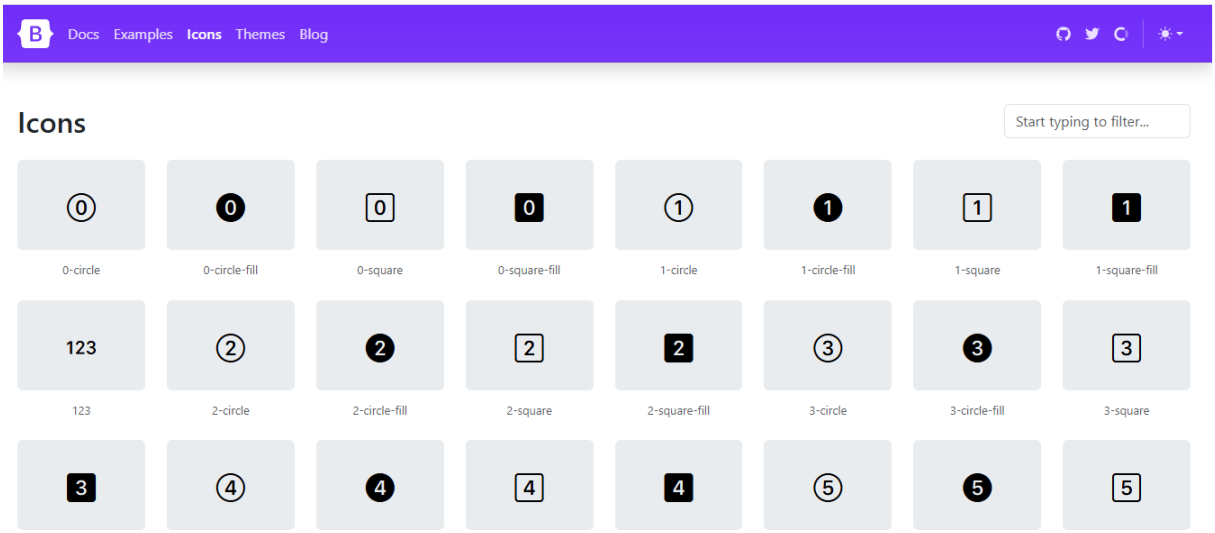
- Afficher toutes les informations d'un article
- Utiliser get_object_or_404

```
def details_article(request, id):
    article = get_object_or_404(Article, id=id)
    return render(request, 'details_article.html', {'article': article})
```

7. Templates HTML:
créer les fichiers suivants :

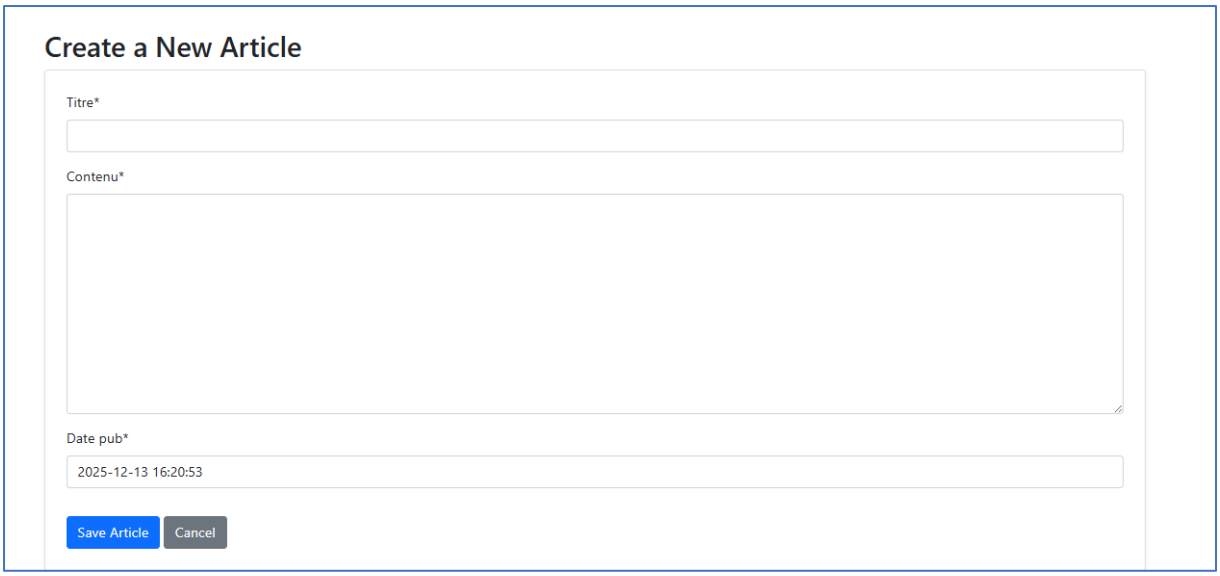
a) liste_articles.html

❖ Pour ajouter des **icônes** dans notre projet, il faut inclure la bibliothèque **Bootstrap Icons**.
Dans ton index.html (ou le template de base), ajoute le lien suivant dans la section <head> :
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.1/font/bootstrap-icons.css" rel="stylesheet">

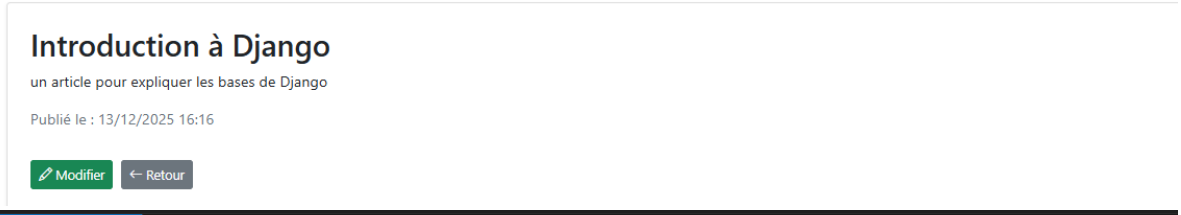


b) create_articles.html

```
{% extends "index.html" %}

{% block title %}Create Article{% endblock %}

{% block content %}
{% load crispy_forms_tags %}

<div class="container mt-4">
    <h2>Create a New Article</h2>

    <form method="POST" class="card p-4 shadow-sm">
        {% csrf_token %}
        {{ form|crispy }}

        <div class="mt-3">
            <button type="submit" class="btn btn-primary">Save Article</button>
            <a href="{% url 'liste_articles' %}" class="btn btn-secondary">Cancel</a>
        </div>
    </form>
</div>

{% endblock %}
```



c) details_article.html



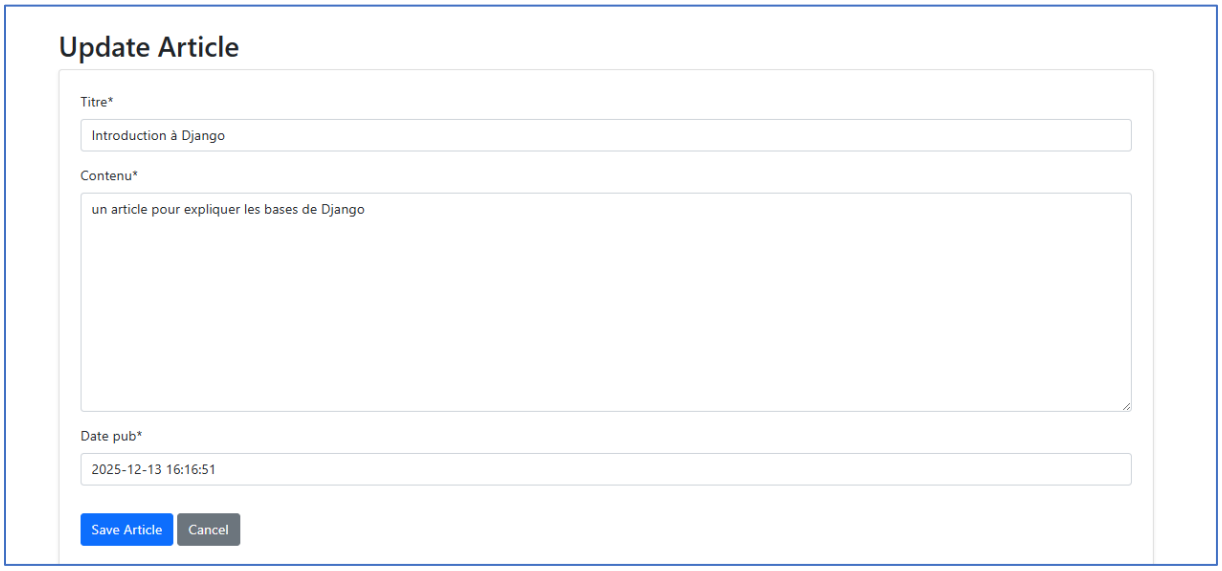
details_article.html

FirstApp > templates > details_article.html > ...

```html
{% extends 'index.html' %}

{% block title %}Détails Article - {{ block.super }}{% endblock %}

{% block content %}
<div class="container mt-4">
    <div class="card shadow-sm p-4">
        <h2 class="card-title">{{ article.titre }}</h2>
        <p class="card-text">{{ article.contenu }}</p>
        <p class="text-muted">Publié le : {{ article.date_pub|date:"d/m/Y H:i" }}</p>

        <div class="d-flex gap-2 mt-3">
            <a href="{% url 'update_articles' article.id %}" class="btn btn-success btn-sm d-flex align-items-center">
                <i class="bi bi-pencil me-1"></i> Modifier
            </a>
            <a href="{% url 'liste_articles' %}" class="btn btn-secondary btn-sm d-flex align-items-center">
                <i class="bi bi-arrow-left me-1"></i> Retour
            </a>
        </div>
    </div>
</div>
{% endblock %}
```

d) update_articles.html

update_articles.html

FirstApp > templates > update_articles.html > ...

```html
{% extends "index.html" %}

{% block title %}Update Article{% endblock %}

{% block content %}
{% load crispy_forms_tags %}

<div class="container mt-4">
    <h2>Update Article</h2>

    <form method="POST" class="card p-4 shadow-sm">
        {% csrf_token %}
        {{ form|crispy }}

        <div class="mt-3">
            <button type="submit" class="btn btn-primary">Save Article</button>
            <a href="{% url 'liste_articles' %}" class="btn btn-secondary">Cancel</a>
        </div>
    </form>
</div>

{% endblock %}
```